Dynamic Portfolio Optimization with a Time-Varying Equity Universe

TEAM NAPPERS

Abstract

This paper develops and studies a rolling portfolio optimization framework for U.S. equities using a dynamic investable universe and a long-only mean–variance objective. At each rebalancing date, the set of eligible stocks is formed using only information available at that time, requiring a minimum trailing return history in order to reduce look-ahead and survivorship concerns. Expected returns and covariance matrices are estimated from rolling historical windows, and portfolio weights are chosen by solving a constrained optimization problem that trades off expected return, portfolio variance, and turnover relative to the previous allocation. The optimization is implemented in PyTorch using a softmax parameterization to enforce full investment and nonnegativity of weights. Using a CRSP-style monthly stock panel, the paper evaluates the resulting strategy in a fully out-of-sample backtest against an equal-weight benchmark formed over the same date-specific universe. Under a baseline specification, the optimized portfolio delivers performance very similar to the benchmark, with only modest improvements in volatility and Sharpe ratio and little visible separation in cumulative equity. Diagnostic exercises show, however, that this near-equal-weight behavior is not due to a coding failure. In simplified two-asset tests, the optimizer tilts toward the higher-return asset as expected, and under more aggressive optimization settings it moves strongly away from equal weight. Motivated by these diagnostics, the paper then studies a less restrictive specification with a weaker turnover penalty and a longer optimization run. Under this alternative specification, the portfolio departs more materially from equal weight and achieves higher cumulative return, though with slightly higher volatility and only negligible improvement in risk-adjusted performance. The results show that the framework is internally coherent and capable of producing active portfolio tilts, but that its out-of-sample success depends critically on the interaction between return estimation, risk penalization, turnover control, and optimization settings. The paper therefore contributes a clean baseline implementation of dynamic portfolio choice and highlights the practical difficulty of generating economically meaningful improvement over a simple equal-weight benchmark.

1 Introduction

Portfolio choice is one of the central problems in financial economics. In its classical form, the investor seeks to allocate wealth across risky assets in a way that balances expected return against risk. The canonical mean–variance framework provides a natural starting point for this problem, but its practical implementation is often difficult. Out-of-sample portfolio performance depends not only on the optimization problem itself, but also on how expected returns and covariance matrices are estimated, how the investable universe is defined, how frequently the portfolio is rebalanced, and how trading frictions are controlled. As a result, even theoretically appealing portfolio rules often perform only modestly better than simple benchmarks in real data.

This paper develops and studies a dynamic portfolio optimization pipeline for U.S. equities using a CRSP-style monthly stock panel. At each rebalancing date, the investor chooses a long-only portfolio over a time-varying set of eligible stocks. The optimization problem trades off three

considerations: expected return, variance, and turnover relative to the previous portfolio. Expected returns and covariance matrices are estimated from rolling windows of past returns, and the constrained optimization problem is solved numerically in PyTorch. The framework is intentionally simple in its economic ingredients, but careful in its implementation. In particular, the portfolio is formed using only information available at the time of decision, and the investable universe is allowed to vary over time in order to avoid relying on full-sample stock availability.

A central issue in empirical portfolio construction is that apparently sophisticated strategies can collapse in practice to something very close to equal weighting. This can happen for several reasons. Expected return estimates may be too noisy to generate strong cross-sectional signals. Risk penalties may dominate the return term. Turnover penalties may anchor the solution too tightly to the prior portfolio. Numerical optimization may also fail to move far from initialization if the search is too conservative. For these reasons, it is not enough to report backtest performance alone. One must also verify that the optimizer is behaving as intended and that the resulting portfolios are genuinely distinct from a naive benchmark.

The contribution of this paper is therefore twofold. First, it provides a clean and internally consistent implementation of rolling mean–variance portfolio choice with a dynamic equity universe, long-only simplex constraints, and explicit turnover control. Second, it uses a series of diagnostic exercises to distinguish between economic limitations of the model and mechanical failures of the implementation. In the baseline specification, the optimized portfolio performs very similarly to an equal-weight benchmark over the same date-specific universe. However, simplified two-asset sanity checks show that the optimizer does in fact tilt toward higher-return assets when the signal is strong. Moreover, when the turnover penalty is weakened and the optimization routine is allowed to run longer, the full backtest begins to separate more clearly from equal weight and achieves higher cumulative return, though with slightly higher volatility and little change in Sharpe ratio.

The main conclusion is that the framework is working as designed, but that economically meaningful improvement over equal weight is difficult to obtain in this setting. The baseline specification is highly conservative and produces only small deviations from the benchmark. A more aggressive specification generates more active portfolios and stronger cumulative performance, but at the cost of higher risk and greater turnover. These findings underscore a broader lesson in applied portfolio optimization: success depends not just on writing down an objective function, but on the interaction between estimation, constraints, penalties, and numerical implementation.

The rest of the paper proceeds as follows. Section 2 describes the data and the construction of the dynamic investable universe. Section 3 presents the portfolio optimization framework and implementation details. Section 4 explains the rolling backtest design and evaluation metrics. Section 5 reports the baseline empirical results. Section 6 presents sanity checks and an alternative, more aggressive specification. Section 7 concludes.

2 Data and Dynamic Universe Construction

The empirical analysis uses a CRSP-style monthly stock panel containing security identifiers, returns, and standard classification variables. The raw dataset spans 300 monthly dates and 971 distinct securities after initial filtering. Because the goal is to study a realistic portfolio formation problem rather than an idealized balanced panel, the analysis begins by cleaning the stock-level data and then constructing a time-varying investable universe using only information available at each point in time.

We first convert dates into a standard datetime format and sort the data by date and security identifier. We then apply standard CRSP-style screens to retain ordinary common shares listed on

the major U.S. exchanges. In particular, when available, we keep only securities with share codes 10 or 11 and exchange codes 1, 2, or 3. Observations with missing returns are removed, and the resulting stock-level data are reshaped into a monthly return panel with dates as rows and securities as columns. After filtering, the raw return panel has dimension 300×971, where rows correspond to months and columns correspond to distinct securities.

A central design choice in the paper is to avoid defining the portfolio universe using full-sample stock availability. Doing so would risk introducing look-ahead and survivorship bias, since firms with long uninterrupted histories are more likely to be persistent survivors. Instead, the investable universe is constructed dynamically. At each month t, a stock is considered eligible only if it has accumulated at least 60 prior monthly return observations by that date. This rule ensures that each included asset has enough historical data to support estimation of expected returns and covariances, while also preventing future information from affecting current portfolio eligibility.

To keep the optimization problem tractable and comparable over time, the number of eligible stocks is capped at 100 each month. In practice, after the initial burn-in period required to accumulate 60 months of return history, the eligible universe reaches this 100-stock cap for most of the sample. The average universe size is 80, the median is 100, and the upper quartiles are also 100, reflecting the fact that the universe is empty in the earliest years and then stabilizes at the cap for much of the backtest. The first nonempty universes appear in early 2005, which is consistent with the 60-month history requirement.

This dynamic-universe construction serves two purposes. First, it makes the backtest more realistic by requiring portfolio eligibility to be determined using only contemporaneously available information. Second, it improves estimation reliability by excluding stocks with insufficient history. The cost is that the set of investable assets changes over time, so the portfolio problem is no longer defined on a fixed cross section of stocks. That complication is addressed explicitly in the optimization and backtesting procedures described below.

Overall, this data construction step yields a panel that is large enough to support a meaningful monthly portfolio problem while remaining disciplined about information timing. It also provides a clean foundation for interpreting the later results, since any deviations from benchmark performance can be attributed to the optimization framework itself rather than to leakage from full-sample stock selection.

3 Portfolio Optimization Framework

At each rebalancing date t, the investor selects a vector of portfolio weights w_t over the currently eligible stock universe. The portfolio is constrained to be long only and fully invested, so the weights must satisfy

$$w_{i,t} \geq 0 \quad \text{for all } i, \qquad \sum_{i=1}^{N_t} w_{i,t} = 1,$$

where N_t denotes the number of eligible assets at date t. Within this feasible set, the portfolio is chosen to trade off expected return, variance, and turnover relative to the previous allocation.

The objective function is

$$\max_{w_t} \; \mu_t^\top w_t \; - \; \gamma \, w_t^\top \Sigma_t w_t \; - \; \kappa \| w_t - w_{t-1} \|_1 .$$

Here, μ_t is the vector of estimated expected returns, Σ_t is the estimated covariance matrix of returns, $\gamma > 0$ is the risk-aversion parameter, and $\kappa > 0$ is the turnover penalty parameter. The first term rewards assets with high estimated returns, the second penalizes portfolio variance, and the third

discourages large reallocations relative to the previous month's holdings. This turnover term is important because, in dynamic portfolio problems, unconstrained reoptimization can otherwise generate unstable and excessively reactive allocations.

The moment estimates are constructed from a trailing rolling window of monthly returns. Specifically, at each date t, the model uses the previous 24 months of returns for the currently eligible assets to estimate

$$\mu_t = \frac{1}{L} \sum_{\ell=1}^{L} r_{t-\ell}$$

and

$$\Sigma_t = \frac{1}{L-1} \sum_{\ell=1}^{L} (r_{t-\ell} - \mu_t)(r_{t-\ell} - \mu_t)^\top,$$

where $L = 24$ is the rolling estimation window. In implementation, the sample covariance matrix is symmetrized and a small diagonal perturbation is added:

$$\Sigma_t \leftarrow \frac{1}{2}(\Sigma_t + \Sigma_t^\top) + \varepsilon I,$$

with ε set to a very small positive constant. This step improves numerical stability and helps prevent singularity or near-singularity in the covariance estimate.

The optimization is implemented in PyTorch. Rather than solving the constrained problem directly, the portfolio weights are parameterized through an unconstrained vector of logits v_t, with weights defined by the softmax map

$$w_{i,t} = \frac{\exp(v_{i,t})}{\sum_{j=1}^{N_t} \exp(v_{j,t})}.$$

This transformation guarantees by construction that all weights are nonnegative and sum to one, so the long-only simplex constraints are always satisfied during optimization. The objective is then optimized over the logits using gradient-based updates.

An additional complication arises because the asset universe changes over time. Since some stocks enter or leave the eligible set from one month to the next, the previous portfolio w_{t-1} cannot simply be reused as a fixed vector. Instead, the previous weights are aligned to the current asset universe by matching on security identifiers. Any stock not held previously is assigned prior weight zero, and the aligned vector is renormalized when necessary. If no prior portfolio exists, the optimizer is initialized at equal weight. This alignment step ensures that the turnover penalty is economically meaningful even when the investable universe changes across months.

In the baseline specification, the model uses a rolling window of 24 months, risk aversion $\gamma = 5.0$, turnover penalty $\kappa = 0.1$, learning rate 0.1, and 200 gradient steps per rebalance. These values are intentionally conservative and serve as a benchmark specification rather than as a fully tuned trading rule. Later sections show that the empirical behavior of the strategy is sensitive to these choices, especially the turnover penalty and the aggressiveness of the numerical optimization routine.

This framework is deliberately simple in its economic structure but careful in its implementation. Its purpose is not to claim that sample mean–variance estimates are an ideal forecasting model, but rather to build a coherent and leak-free baseline for studying how dynamic portfolio optimization behaves in practice.

4 Backtest Design and Performance Evaluation

The strategy is evaluated in a rolling out-of-sample backtest. At each rebalancing date t, the eligible stock universe is first identified using only information available at that date. Using the previous 24 months of returns for those eligible assets, the model estimates the mean return vector and covariance matrix, aligns the previous portfolio to the current stock set, and solves the constrained optimization problem to obtain a new vector of portfolio weights w_t. These weights are then held over the next month, and the realized portfolio return is computed using returns in period $t+1$.

To ensure that estimation and evaluation are both well defined, the code imposes two additional data requirements at each rebalance step. First, an asset must have a complete return history over the 24-month estimation window in order to remain in the optimization problem. Second, the asset must also have an observed realized return in the next month so that the out-of-sample portfolio return can be computed. Assets failing either condition are excluded from the portfolio at that date. This means that the effective investment universe may be smaller than the initial eligible set, but it ensures that each backtest observation is constructed cleanly and without missing data contamination.

Because the investable universe changes over time, the benchmark portfolio is also defined dynamically. Rather than comparing the optimized strategy to a fixed market index, the paper uses an equal-weight benchmark formed over the same set of assets that are available to the optimizer at each date. This makes the comparison more transparent: both portfolios face the same stock universe in each month, and any performance difference can therefore be attributed to the portfolio weighting rule rather than to differential asset availability.

The full backtest produces 239 out-of-sample monthly observations for both the optimized strategy and the equal-weight benchmark. This sample length is consistent with the overall 300-month panel once one accounts for the initial history requirement for universe formation, the 24-month rolling estimation window, and the need for a subsequent month of realized returns. The resulting return series provide the basis for all subsequent performance comparisons and diagnostics.

Portfolio performance is evaluated using several standard summary statistics. Let r_t^p denote the realized return on a portfolio in month t. The mean monthly return is

$$\bar{r}^p = \frac{1}{T}\sum_{t=1}^{T} r_t^p,$$

and annualized volatility is computed as

$$\sigma_{\text{ann}} = \sqrt{12}\,\text{sd}(r_t^p),$$

where $\text{sd}(r_t^p)$ is the sample standard deviation of monthly returns. The annualized Sharpe ratio is defined as

$$\text{Sharpe} = \frac{\sqrt{12}\,\bar{r}^p}{\text{sd}(r_t^p)},$$

using raw returns rather than excess returns, since the purpose here is primarily comparative.

In addition to mean and volatility measures, the paper tracks cumulative performance and drawdowns. The equity curve is constructed recursively as

$$E_t = \prod_{s=1}^{t}(1 + r_s^p),$$

so that E_t represents the growth of one dollar invested in the strategy. Cumulative return is then $E_T - 1$. Maximum drawdown is computed from the equity curve as the most negative value of

$$\frac{E_t}{\max_{s \le t} E_s} - 1,$$

which measures the largest peak-to-trough decline experienced during the backtest.

These performance measures serve two purposes. First, they summarize the economic behavior of the strategy in terms of return, risk, and cumulative growth. Second, they allow direct comparison between the optimized portfolio and the equal-weight benchmark under both the baseline and alternative specifications. Because the benchmark is constructed over the same date-specific universe, these comparisons are especially useful for determining whether the optimization procedure adds value beyond a simple naive allocation rule.

5 Baseline Results

We begin with the baseline specification of the model, which uses a 24-month rolling estimation window, risk aversion $\gamma = 5.0$, turnover penalty $\kappa = 0.1$, learning rate 0.1, and 200 gradient steps per rebalance. Under this specification, the optimized portfolio and the equal-weight benchmark produce very similar out-of-sample performance. This is already informative. It suggests that, in the baseline setting, the optimization either does not generate large cross-sectional tilts or that any such tilts are too weak to produce a visibly different equity path from the benchmark.

The optimized portfolio achieves a mean monthly return of 0.0092, annualized volatility of 0.1415, annualized Sharpe ratio of 0.7774, maximum drawdown of -0.4668, and cumulative return of 6.2591. Over the same dynamic universe, the equal-weight benchmark delivers a mean monthly return of 0.0094, annualized volatility of 0.1476, annualized Sharpe ratio of 0.7620, maximum drawdown of -0.4664, and cumulative return of 6.4788. These numbers indicate that the optimized portfolio provides slightly lower volatility and a marginally higher Sharpe ratio, but it does not outperform the benchmark in cumulative return and does not materially improve drawdown behavior.

The cumulative equity curves reinforce this conclusion. Visually, the optimized portfolio and the equal-weight benchmark track each other very closely over the full sample, with only small and transitory differences. The optimized strategy does not generate a clearly distinct return path, and in practical terms its behavior under the baseline specification is very similar to that of a naive equal-weight rule. This is a central empirical fact of the paper: the baseline optimization framework is internally coherent, but in its conservative form it does not yet produce a substantial economic improvement over a simple benchmark.

To better understand why the optimized portfolio behaves so similarly to equal weight, we examine portfolio-level diagnostics. Using a time-by-stock weight matrix constructed over the union of all assets that enter the backtest, the average turnover between consecutive portfolio rebalances is only 0.0052. This is extremely low and indicates that the portfolio changes very little from month to month. Such low turnover is consistent with the relatively strong turnover penalty in the baseline objective and suggests that the optimizer is being heavily anchored to its previous allocation. The strategy is therefore dynamic in principle, but quite static in practice.

Taken together, the baseline performance results imply that the model is not obviously failing, but that it is operating in a very conservative regime. The optimized portfolio is slightly smoother than the benchmark and modestly better on a risk-adjusted basis, but the differences are too small to claim a meaningful gain in economic performance. This motivates the next part of the analysis, which asks whether the optimizer is genuinely capable of deviating from equal weight and

whether the baseline closeness to the benchmark reflects implementation error, overly restrictive hyperparameters, or simply weak return signals in the data.

6 Sanity Checks and Diagnostics

Because the baseline backtest produces results very close to the equal-weight benchmark, it is important to determine whether this reflects an economic feature of the specification or a flaw in the implementation. In particular, a near-equal-weight outcome could arise for at least three distinct reasons. First, the optimizer might be coded incorrectly or fail to respect the intended objective. Second, the numerical optimization routine might be too weak to move far from its initialization. Third, the combination of risk aversion, turnover penalization, and noisy expected-return estimates might simply make the optimal portfolio genuinely close to equal weight. This section addresses these possibilities directly.

We begin with a simple two-asset diagnostic constructed from the full return panel. Among stocks with sufficient history, we search for a pair consisting of one asset with a relatively high average realized monthly return and one asset with a relatively low average realized monthly return, subject to the requirement that the pair must have at least one full rolling estimation window of overlapping observations. This yields a "good" stock with an average monthly return of 0.0542 and a "bad" stock with an average monthly return of -0.0271, with 34 months of overlap. The purpose of this exercise is not to produce an economically realistic trading rule, but to create a simple environment in which the optimizer should clearly prefer one asset to the other if it is functioning correctly.

Using the last 24 overlapping months of returns for this pair, we estimate the corresponding mean vector and covariance matrix and solve the same constrained optimization problem used in the main backtest. To isolate the behavior of the optimizer itself, we temporarily set the turnover penalty to zero and initialize the portfolio from scratch rather than from previous holdings. Under these conditions, the estimated mean returns are 0.0472 for the good stock and -0.0022 for the bad stock, and the optimizer assigns weights of 0.6089 and 0.3911, respectively. This is an important result. The optimizer does not remain at equal weight, but instead tilts toward the asset with the stronger expected return. That finding rules out the simplest implementation failure: the optimization routine is not mechanically producing equal-weight portfolios.

We next relax the problem further by lowering the risk-aversion parameter from $\gamma = 5.0$ to $\gamma = 1.0$, again with the turnover penalty set to zero. Under this weaker penalty structure, the optimizer shifts slightly further toward the better asset, increasing its weight from 0.6089 to 0.6170. Although the change is modest, it is in the theoretically correct direction. This confirms that the optimizer is responsive to the penalty parameters and that its behavior changes sensibly when the objective is made less conservative.

A final and stronger diagnostic examines the role of numerical convergence. In the same two-asset setting, we keep the relaxed penalties, but now increase the learning rate substantially and raise the number of gradient steps from 200 to 2000. Under these more aggressive numerical settings, the optimizer assigns weight 0.9943 to the good stock and only 0.0057 to the bad stock. Moreover, the objective value rises from 0.0150 at equal weight to 0.0333 at the optimized solution. This result is especially revealing. It shows not only that the optimizer can deviate strongly from equal weight, but also that doing so produces a substantially better value of the model's own objective function. In other words, the routine is capable of converging to a highly tilted portfolio when the underlying signal is strong and the search is given sufficient room to proceed.

These diagnostic exercises lead to a clear conclusion. The baseline model's similarity to equal

weight is not the product of a coding mistake. The objective function is implemented correctly, the simplex constraints are being enforced properly, and the gradient-based optimization routine can find strongly non-equal solutions in simplified environments. Instead, the baseline behavior arises from a combination of conservative penalty choices and limited optimizer movement in the full many-asset problem. This is precisely the kind of distinction that must be established before interpreting the economic meaning of the backtest results.

The diagnostics also suggest a natural next step. If the baseline specification is too conservative, then a more aggressive version of the same model should generate larger deviations from equal weight and potentially a more distinct performance profile. The following section therefore studies an alternative specification with a much weaker turnover penalty and a longer optimization run in order to determine whether the full strategy can be moved out of its near-equal-weight regime.

7 Aggressive Specification and Alternative Results

Motivated by the diagnostic results above, we next consider a less conservative version of the portfolio optimization problem. The goal is to determine whether the baseline model's closeness to equal weight is an unavoidable feature of the data or instead a consequence of restrictive penalty choices and limited optimizer movement. To study this question, we rerun the full rolling backtest under a more aggressive specification that weakens the turnover penalty and allows the numerical optimization routine to search more extensively.

Specifically, the alternative specification keeps the risk-aversion parameter fixed at $\gamma = 5.0$, but reduces the turnover penalty from $\kappa = 0.1$ to $\kappa = 0.005$ and increases the number of gradient steps from 200 to 1000. The learning rate remains at 0.1. All other aspects of the empirical design remain unchanged. The dynamic stock universe is constructed in the same way, the moment estimates are still based on the previous 24 months of returns, and the benchmark remains the equal-weight portfolio over the same date-specific asset set. As in the baseline case, the rerun produces 239 out-of-sample monthly observations.

Under this more aggressive specification, the optimized portfolio begins to separate more clearly from the equal-weight benchmark. The strategy achieves a mean monthly return of 0.0096, annualized volatility of 0.1514, annualized Sharpe ratio of 0.7626, maximum drawdown of -0.4752, and cumulative return of 6.8492. The equal-weight benchmark over the same dynamic universe delivers a mean monthly return of 0.0094, annualized volatility of 0.1476, annualized Sharpe ratio of 0.7620, maximum drawdown of -0.4664, and cumulative return of 6.4788. Relative to the baseline specification, the aggressive portfolio therefore achieves a visibly higher cumulative return and a somewhat higher mean monthly return, but at the cost of higher volatility and a slightly worse drawdown profile. The Sharpe ratio improves only negligibly.

The cumulative equity curves reinforce this interpretation. Unlike in the baseline specification, where the optimized portfolio and the benchmark were nearly indistinguishable, the aggressive portfolio now begins to pull away from equal weight, especially in the later part of the sample. This indicates that once the turnover penalty is weakened and the optimizer is allowed to search longer, the model can generate a more distinct return path. At the same time, the improvement is not uniform across all performance dimensions. The strategy is no longer merely a smoother version of equal weight; it is taking more active risk, and its stronger cumulative growth reflects that fact.

This interpretation is confirmed by the portfolio diagnostics. Under the aggressive specification, the average L^1 distance from the month-specific equal-weight portfolio is 0.9049, and the average monthly turnover rises to 0.0289. Both numbers are much larger than in the baseline specification, where turnover was only 0.0052 and the portfolio remained visually close to the benchmark. The

aggressive model is therefore not only producing better cumulative return, but also behaving in a fundamentally more active way. It is departing more meaningfully from equal weight and updating its holdings more substantially through time.

These results are important for two reasons. First, they show that the baseline similarity to equal weight was not an inherent consequence of the data alone. The same underlying framework can produce a materially different strategy once the turnover penalty and optimization settings are changed. Second, they illustrate the tradeoff at the heart of practical portfolio optimization. Greater deviation from equal weight can raise cumulative return, but it also tends to increase volatility, turnover, and sensitivity to estimation error. The aggressive specification therefore improves raw performance, but it does not deliver a decisive improvement in risk-adjusted terms.

Taken together, the alternative results suggest that the model is capable of generating active portfolio tilts, but that the economic value of those tilts remains limited. The framework is not stuck mechanically at equal weight, yet moving away from equal weight does not automatically translate into a large Sharpe improvement. This is a useful empirical finding in its own right. It shows that the main challenge in this setting is not simply optimization, but rather the difficulty of extracting a strong enough return signal from rolling sample moments to overcome noise, transaction considerations, and risk penalties in a robust way.

8 Conclusion

This paper develops and studies a dynamic mean–variance portfolio optimization framework for monthly U.S. equity data. The model combines a time-varying investable universe, rolling estimation of expected returns and covariance matrices, long-only simplex constraints, and an explicit turnover penalty. Implemented in PyTorch, the framework is designed to be internally coherent, leak-free, and flexible enough to accommodate changing asset availability over time. The central empirical question is whether such a framework can produce a portfolio that meaningfully outperforms a simple equal-weight benchmark in out-of-sample backtesting.

Under the baseline specification, the answer is largely negative. The optimized portfolio performs very similarly to the equal-weight benchmark, with only modest differences in volatility and Sharpe ratio and little visible divergence in cumulative equity. Portfolio diagnostics show that turnover is extremely low in this regime, indicating that the optimizer is being heavily anchored to its previous allocation. On its face, this could suggest either that the optimization is failing or that the chosen specification is simply too conservative to generate meaningful active tilts.

The diagnostic exercises strongly support the latter interpretation. In a two-asset test with clearly different return behavior, the optimizer assigns greater weight to the higher-return asset rather than defaulting to equal weight. When the penalty structure is relaxed and the numerical optimization is made more aggressive, the same routine converges to a highly concentrated solution with a substantially better objective value than equal weight. These results confirm that the code is functioning properly and that the baseline closeness to equal weight is not a mechanical artifact of the implementation.

Motivated by these diagnostics, the paper then studies a more aggressive full-sample specification with a much weaker turnover penalty and a longer optimization run. Under this alternative setting, the strategy departs more materially from equal weight, exhibits higher turnover, and achieves a higher cumulative return than the benchmark. At the same time, volatility rises, drawdowns become slightly worse, and the Sharpe ratio improves only negligibly. Thus, the more aggressive specification demonstrates that the framework can generate active portfolio behavior, but it also shows that stronger portfolio tilts do not automatically translate into materially better risk-adjusted

performance.

The broader lesson is that practical portfolio optimization is shaped as much by estimation and implementation choices as by the formal objective function itself. In this setting, the main challenge is not simply solving the optimization problem, but doing so in a way that extracts a sufficiently strong and stable return signal from noisy rolling sample moments. Conservative penalties can make the strategy behave almost like equal weight, while aggressive penalties can produce more active portfolios without necessarily delivering a large Sharpe improvement. The tension between responsiveness and robustness is therefore central.

Several extensions suggest themselves for future work. On the estimation side, one could replace sample means and sample covariances with shrinkage estimators, factor-model forecasts, or machine-learning based return predictors. On the optimization side, one could introduce explicit transaction cost models, exposure constraints, or alternative risk measures. On the empirical side, one could compare performance across different universe definitions, rebalancing frequencies, or benchmark constructions. The current paper does not resolve these broader questions, but it establishes a clean baseline framework in which they can be studied.

In that sense, the main contribution of the paper is methodological as much as empirical. It provides a transparent implementation of dynamic portfolio choice under realistic information timing, shows how to diagnose near-equal-weight outcomes, and illustrates the practical difficulty of improving on a naive benchmark in out-of-sample equity allocation. That is a useful benchmark result in itself, and it provides a foundation for richer future models.

9 Data, Universe Construction, and Portfolio Optimization Procedure

This section describes the data, preprocessing, portfolio formation procedure, optimization problem, backtesting design, and hyperparameter search implemented in the code. The goal is to make explicit every substantive modeling and numerical decision embedded in the empirical procedure.

9.1 Data and Raw Panel Structure

The input dataset is a stock-level monthly panel stored as a CSV file. Each row corresponds to a particular stock in a particular month. The key variables used in the code are:

- `date`: the calendar month of the observation,
- `permno`: a stock identifier,
- `ret`: the stock's monthly return,
- `shrcd`: a CRSP-style share code, when available,
- `exchcd`: a CRSP-style exchange code, when available,
- `prc`: the stock price, when available.

The code first converts `date` into a datetime variable and sorts the data by `date` and `permno`. It then applies standard stock-level filters when the relevant columns are present. Specifically, if `shrcd` is available, only observations with `shrcd` in $\{10, 11\}$ are retained, thereby restricting attention to common shares. If `exchcd` is available, only observations with `exchcd` in $\{1, 2, 3\}$ are retained, thereby restricting attention to stocks listed on the major U.S. exchanges. Returns are coerced to numeric values, and all observations with missing or nonnumeric returns are dropped.

After filtering, the code reshapes the stock-level panel into a date-by-asset return matrix,

$$R = \{R_{t,i}\}_{t=1,\dots,T;\, i=1,\dots,N},$$

where $R_{t,i}$ denotes the monthly return of stock i in month t. This matrix is constructed by pivoting the data so that rows correspond to dates, columns correspond to `permno` values, and entries correspond to returns. When price data are available, the code also constructs an analogous date-by-asset price matrix using the absolute value of `prc`, which is standard when raw CRSP prices may carry sign conventions unrelated to economic price levels.

9.2 Dynamic Universe Construction

The portfolio is not formed over a fixed universe. Instead, the code constructs a *date-specific eligible universe* at each month t. This universe depends only on information available on or before date t, thereby avoiding look-ahead bias.

Three user-specified parameters govern universe formation:

$$\begin{aligned} \texttt{MIN_HISTORY} &= 60, \\ \texttt{K_UNIVERSE} &= 350, \\ \texttt{MIN_PRICE} &= 0.1. \end{aligned}$$

History filter. A stock is considered eligible at date t only if it has at least 60 observed monthly returns in the preceding 60 months. Formally, if $I_{s,i} = 1$ when stock i has a valid return in month s, and 0 otherwise, then stock i is eligible at month t only if

$$\sum_{s=t-60}^{t-1} I_{s,i} \geq 60.$$

The code implements this with a rolling count shifted by one period, so the eligibility decision at date t uses only information from months strictly prior to t.

Price filter. If price data are available, then a stock passing the history filter must additionally satisfy

$$|P_{t,i}| \geq 0.1,$$

where $P_{t,i}$ denotes the stock's recorded price at date t. This excludes extremely low-priced securities.

Universe truncation. Among stocks passing the eligibility rules at date t, the code retains at most 350 assets. Importantly, this truncation is not based on market capitalization, liquidity, return, or any other economically ranked criterion. Instead, the code simply keeps the first 350 eligible assets in the column ordering of the return panel. This is an implementation decision rather than a substantive asset-selection rule and should be interpreted as such.

Thus, the investable universe $\mathcal{U}_t$ at date t is a dynamic set determined by prior return availability, an optional price screen, and a hard cap on the number of admissible assets.

9.3 Rolling Estimation Window

At each rebalance date t, the portfolio is formed using a rolling estimation window of length

$$\texttt{ROLLING_WINDOW} = 24.$$

Accordingly, moments are estimated from the 24 most recent months strictly prior to date t, namely from the set of dates

$$\{t-24, t-23, \ldots, t-1\}.$$

Let $\mathcal{U}_t$ denote the eligible universe at date t. The code extracts the submatrix of returns over the past 24 months for the stocks in $\mathcal{U}_t$. It then applies an additional missing-data filter: any stock with even a single missing return in the 24-month estimation window is removed from the current optimization problem. Therefore, the actual optimization universe at date t, which we denote by $\widetilde{\mathcal{U}}_t$, satisfies

$$\widetilde{\mathcal{U}}_t \subseteq \mathcal{U}_t,$$

with inclusion possibly strict whenever an otherwise eligible stock has incomplete data in the most recent estimation window.

This implies two separate data-completeness requirements:

1. A stock must have at least 60 prior monthly returns to be considered eligible.
2. Among those eligible stocks, only those with complete data in the most recent 24-month estimation window are retained for optimization.

9.4 Moment Estimation

Given the rolling return matrix $R_t \in \mathbb{R}^{24\times N_t}$, where $N_t = |\widetilde{\mathcal{U}}_t|$, the code estimates expected returns and covariance by simple sample moments.

Expected return estimator. For each asset $i \in \widetilde{\mathcal{U}}_t$, the estimated expected return is the sample mean over the 24-month window:

$$\hat{\mu}_{t,i} = \frac{1}{24} \sum_{s=t-24}^{t-1} R_{s,i}.$$

Collecting these across assets yields the vector

$$\hat{\mu}_t \in \mathbb{R}^{N_t}.$$

Covariance estimator. The covariance matrix is estimated as the sample covariance matrix of the same 24-month window:

$$\hat{\Sigma}_t \in \mathbb{R}^{N_t\times N_t}.$$

Two numerical stabilization steps are then applied:

1. *Symmetrization:*

$$\hat{\Sigma}_t \leftarrow \frac{1}{2}\left(\hat{\Sigma}_t + \hat{\Sigma}_t^\top\right).$$

   This ensures exact symmetry in case of floating-point asymmetries.

2. *Diagonal regularization:*

$$\hat{\Sigma}_t \leftarrow \hat{\Sigma}_t + \varepsilon I, \qquad \varepsilon = 10^{-6}.$$

   This adds a small ridge term for numerical stability.

Hence, the code implements a rolling sample mean–sample covariance framework with a light positive-definiteness regularization.

9.5 Previous Portfolio Alignment Under a Changing Universe

Because the asset universe changes over time, the code cannot directly impose a turnover penalty against the previous weight vector without reconciling the prior portfolio with the current universe.

Let w_{t-1} denote the previously chosen portfolio, stored as a mapping from stock identifiers to weights. For the current asset set $\widetilde{\mathcal{U}}_t$, the code constructs an aligned previous portfolio w_{t-1}^{align} by:

1. assigning each currently present stock its prior-period weight if it was previously held,
2. assigning weight 0 to any newly appearing stock.

Formally, for each $i \in \widetilde{\mathcal{U}}_t$,

$$\widetilde{w}_{t-1,i} = \begin{cases} w_{t-1,i}, & \text{if } i \text{ was in the previous portfolio,} \\ 0, & \text{otherwise.} \end{cases}$$

The code then renormalizes this vector over the current universe whenever the remaining mass is positive:

$$w_{t-1,i}^{\text{align}} = \frac{\widetilde{w}_{t-1,i}}{\sum_{j\in\widetilde{\mathcal{U}}_t} \widetilde{w}_{t-1,j}}.$$

If the total surviving mass is zero, the code instead initializes the reference portfolio at equal weights over the current universe:

$$w_{t-1}^{\text{align}} = \left(\frac{1}{N_t}, \ldots, \frac{1}{N_t}\right).$$

This is a substantive modeling decision. The turnover penalty is therefore computed relative not to the full prior portfolio over the old universe, but relative to the renormalized surviving portion of that portfolio restricted to the current asset set.

9.6 Portfolio Optimization Problem

At each rebalance date t, the code solves a long-only, fully invested portfolio optimization problem. Let

$$w_t \in \mathbb{R}^{N_t}$$

denote the vector of portfolio weights over the current universe $\widetilde{\mathcal{U}}_t$. The objective is

$$\max_{w_t \in \Delta^{N_t}} \left\{ \hat{\mu}_t^\top w_t - \Gamma\, w_t^\top \hat{\Sigma}_t w_t - \|w_t - w_{t-1}^{\text{align}}\|_1 \right\},$$

where

$$\Delta^{N_t} = \left\{ w \in \mathbb{R}^{N_t} : w_i \geq 0 \ \forall i, \quad \sum_{i=1}^{N_t} w_i = 1 \right\}$$

is the unit simplex.

The three terms in the objective admit straightforward economic interpretations:

1. $\hat{\mu}_t^\top w_t$: expected portfolio return,
2. $\Gamma\, w_t^\top \hat{\Sigma}_t w_t$: a variance penalty scaled by the risk-aversion parameter Γ,
3. $\|w_t - w_{t-1}^{\text{align}}\|_1$: an ℓ_1 turnover penalty scaled by the parameter .

Thus, the code implements a dynamic mean-variance portfolio problem with a transaction-inertia term.

9.7 Constraint Handling via Softmax Reparameterization

Rather than imposing the simplex constraint through projection or quadratic programming, the code reparameterizes the portfolio weights using an unconstrained vector of logits $v_t \in \mathbb{R}^{N_t}$, with weights defined by the softmax map:

$$w_{t,i} = \frac{\exp(v_{t,i})}{\sum_{j=1}^{N_t} \exp(v_{t,j})}.$$

This construction guarantees that, for every feasible iterate,

$$w_{t,i} > 0 \quad \text{for all } i, \qquad \sum_{i=1}^{N_t} w_{t,i} = 1.$$

Accordingly, the optimization is carried out over unconstrained logits v_t, but the resulting portfolio is always long-only and fully invested. The code therefore rules out short positions, leverage, and cash holdings by construction.

9.8 Numerical Optimization Procedure

The code solves the optimization problem using PyTorch and gradient-based optimization over the logits v_t. If w_{t-1}^{align} is available, the optimizer is initialized near that portfolio by setting

$$v_t^{(0)} = \log\left(w_{t-1}^{\text{align}} + 10^{-8}\mathbf{1}\right).$$

If no previous portfolio exists, then the initial reference portfolio is set to equal weights, and the same log initialization is applied.

The code then uses stochastic gradient descent with learning rate $\eta = \texttt{LR}$, though there is no stochastic subsampling in the implementation, so this is in fact deterministic full-gradient optimization. At each iteration $m = 0, 1, \ldots, \texttt{STEPS} - 1$, the code computes

$$w_t^{(m)} = \text{softmax}(v_t^{(m)}),$$

forms the objective

$$\mathcal{J}(w_t^{(m)}) = \hat{\mu}_t^\top w_t^{(m)} - \Gamma\,(w_t^{(m)})^\top \hat{\Sigma}_t w_t^{(m)} - \|w_t^{(m)} - w_{t-1}^{\text{align}}\|_1,$$

and updates the logits by descending on the loss

$$\mathcal{L}(v_t^{(m)}) = -\mathcal{J}(w_t^{(m)}).$$

The numerical optimizer uses two hyperparameters:

- `LR`: the learning rate,
- `STEPS`: the number of gradient iterations per rebalance date.

This is a crucial point: the code does *not* solve the portfolio problem to an explicit convergence tolerance. Instead, it performs a fixed number of gradient steps and accepts the resulting iterate as the chosen portfolio. Therefore, `LR` and `STEPS` are not merely computational details. They can materially affect the resulting portfolio, especially in parameter regimes where the objective strongly favors concentrated solutions.

9.9 Rebalancing and One-Step-Ahead Evaluation

The backtest proceeds sequentially through time. At each rebalance date t, the code:

1. constructs the eligible universe using only information available at date t,
2. estimates $\hat{\mu}_t$ and $\hat{\Sigma}_t$ using the previous 24 months,
3. aligns the previous portfolio to the current universe,
4. solves the portfolio optimization problem,
5. stores the resulting weight vector w_t,
6. evaluates the realized return of that portfolio using the next month's returns R_{t+1}.

Hence, the realized portfolio return is

$$r_{t+1}^{p} = w_t^{\top} R_{t+1}.$$

A further implementation choice concerns missing next-period returns. If an asset in the chosen portfolio has a missing return in month $t+1$, the code replaces that missing value with zero:

$$R_{t+1,i} \leftarrow 0 \quad \text{whenever } R_{t+1,i} \text{ is missing.}$$

Thus, the evaluation rule treats missing next-period returns as zero realized returns rather than excluding the asset, skipping the observation, or separately incorporating delisting returns. This is a strong backtesting convention and should be stated explicitly.

9.10 Benchmark Construction

For each rebalance date t, the code constructs an equal-weight benchmark over the same date-specific universe used by the optimizer after all missing-data filtering. If $N_t = |\widetilde{\mathcal{U}}_t|$, then the benchmark weight vector is

$$w_t^{\mathrm{EW}} = \left(\frac{1}{N_t}, \ldots, \frac{1}{N_t} \right),$$

and its realized next-period return is

$$r_{t+1}^{\mathrm{EW}} = (w_t^{\mathrm{EW}})^{\top} R_{t+1}.$$

This benchmark is therefore *not* the S&P 500 index, nor a value-weighted market benchmark. Rather, it is a matched-universe equal-weight comparator designed to evaluate whether the optimized weights outperform a naive allocation over the same investable asset set.

9.11 Performance Measurement

After collecting the sequence of realized portfolio returns, the code computes standard performance metrics. Let $\{r_t\}_{t=1}^{T}$ denote a return series measured at monthly frequency.

Mean monthly return.

$$\bar{r} = \frac{1}{T} \sum_{t=1}^{T} r_t.$$

Annualized volatility.

$$\sigma_{\mathrm{ann}} = \sqrt{12}\, \mathrm{sd}(r_t).$$

Annualized Sharpe ratio.

$$\mathrm{Sharpe}_{\mathrm{ann}} = \sqrt{12}\, \frac{\bar{r}}{\mathrm{sd}(r_t)}.$$

No risk-free rate adjustment is applied, so the Sharpe ratio is computed relative to zero.

Equity curve. The growth-of-\$1 series is defined recursively by

$$E_t = \prod_{s=1}^{t} (1 + r_s).$$

Drawdown and maximum drawdown. If

$$M_t = \max_{1 \le s \le t} E_s,$$

then drawdown is

$$D_t = \frac{E_t}{M_t} - 1,$$

and maximum drawdown is

$$\min_{1 \le t \le T} D_t.$$

Total cumulative return.

$$E_T - 1.$$

These statistics are computed both for the optimized portfolio and for the equal-weight benchmark.

9.12 Weight Diagnostics

Beyond return-based evaluation, the code also stores full portfolio weight histories. For each rebalance date, the chosen asset list and associated weight vector are saved. These are then aggregated into a global time-by-asset weight matrix W, where assets absent from the portfolio at a given date receive weight zero.

This allows several additional diagnostics:

1. *Average portfolio weight by asset:* used to identify the stocks receiving the largest average allocations over the backtest.
2. *Average ℓ_1 distance from a global equal-weight vector:* a diagnostic for how far the strategy departs from uniform diversification over the union of all ever-held assets.
3. *Average turnover:* computed as

$$\frac{1}{T-1} \sum_{t=2}^{T} \|W_t - W_{t-1}\|_1.$$

These are descriptive diagnostics rather than direct inputs into the optimization.

9.13 Repeated Re-Runs Under Alternative Hyperparameters

The code re-runs the entire backtest under several alternative parameter settings for

$$(\Gamma, , \texttt{LR}, \texttt{STEPS}).$$

The economic parameters are:

- Γ: penalty on portfolio variance,
- : penalty on turnover.

The numerical parameters are:

- `LR`: gradient step size,

- STEPS: number of gradient updates.

When Γ and are made very small, the objective approaches a constrained expected-return maximization problem:

$$\max_{w \in \Delta^{N_t}} \hat{\mu}_t^\top w.$$

In such regimes, the true optimizer over the simplex would typically be highly concentrated in the assets with the largest estimated means. Because the code solves this approximately using finite-step gradient descent over softmax logits, the final portfolio can be very sensitive to the solver settings. Thus, changes in LR and STEPS can induce real economic changes in the realized allocations.

9.14 Grid Search Over Numerical Hyperparameters

The code performs a grid search over

$$\texttt{LR} \in \{0.0003, 0.001, 0.003, 0.01, 0.03, 0.1\}$$

and

$$\texttt{STEPS} \in \{100, 300, 1000, 3000, 5000\},$$

while holding the economic parameters fixed at aggressive values,

$$\Gamma = 10^{-5}, \qquad = 10^{-5}.$$

For each pair (LR, STEPS), the full backtest is rerun, and performance statistics are recorded. The resulting configurations are ranked primarily by annualized Sharpe ratio and secondarily by cumulative return. The best-performing pair is then selected and assigned back to the global optimization settings.

This search should be interpreted carefully. It is not simply tuning a computational routine to solve a fixed optimization problem more accurately. Because the code does not use a convergence criterion, different values of LR and STEPS can produce substantively different portfolios. Therefore, this grid search is effectively selecting among alternative approximate portfolio policies, not merely among computationally equivalent solvers.

9.15 Grid Search Over Economic Hyperparameters

The code also performs a separate 5×5 grid search over the economic penalty parameters:

$$\Gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\},$$
$$\in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}.$$

During this search, the solver hyperparameters are held fixed at

$$\texttt{LR} = 0.1, \qquad \texttt{STEPS} = 1000.$$

For each parameter pair $(\Gamma,)$, the full backtest is rerun and evaluated. The best pair is selected using the same ranking criterion as above. Heatmaps are then produced for:

- annualized Sharpe ratio,
- Sharpe spread relative to the equal-weight benchmark,
- total cumulative return.

These searches aim to identify economically favorable tradeoffs between expected return, risk control, and turnover control. However, because the numerical solver remains approximate, the selected economic parameters are conditional on the fixed solver settings used during the grid search.

9.16 Interpretation of the Full Procedure

Taken together, the code implements the following empirical design. At each month, it forms a dynamic cross-section of stocks satisfying data-history and optional price requirements, estimates sample expected returns and covariance from the previous 24 months, and chooses a long-only fully invested portfolio by maximizing a mean-variance objective with an additional ℓ_1 penalty on deviations from the previously held portfolio. The optimization is carried out numerically through softmax-reparameterized gradient descent, and the resulting portfolio is evaluated out-of-sample using next-month realized returns. Performance is compared against an equal-weight portfolio over the same date-specific universe.

In short, this is a dynamic-universe, rolling-window, long-only portfolio optimization framework with explicit penalties for risk and turnover, estimated using sample moments and implemented through approximate gradient-based constrained optimization.

9.17 Important Modeling and Implementation Choices

For clarity, the main embedded choices in the code can be summarized as follows:

1. Monthly stock-level data are used.
2. The stock universe is restricted to common shares on major U.S. exchanges when those fields are available.
3. Invalid or missing returns are dropped.
4. Asset eligibility requires at least 60 prior monthly returns.
5. An optional price filter excludes stocks with prices below 0.1.
6. At most 350 assets are retained each month, based on panel ordering rather than an economic ranking rule.
7. Estimation uses a 24-month rolling window.
8. Expected returns are estimated by sample means.
9. Covariances are estimated by sample covariances with symmetrization and ridge regularization.
10. The portfolio is constrained to be long-only and fully invested.
11. Turnover is penalized relative to an aligned and renormalized version of the previous portfolio.
12. Missing next-period returns are treated as zero in evaluation.
13. The benchmark is equal-weight over the same active universe.
14. Optimization is approximate and depends on finite-step gradient descent.
15. Hyperparameter searches are conducted over both economic and numerical parameters.

These details are essential for interpreting the resulting performance statistics and for understanding exactly what empirical object the code is producing.

10 Results

This section reports the empirical behavior of the portfolio optimization procedure under the new set of hyperparameter experiments. The main findings are as follows. First, portfolio performance is highly sensitive to the numerical optimization settings, especially the learning rate and the number of gradient steps. Second, the strongest realized performance occurs in regions of the parameter space where the economic penalties on risk and turnover are very small. Third, under the best-performing specification, the strategy substantially exceeds the terminal wealth of standard passive market benchmarks over the sample period. At the same time, the pattern of results makes clear that this outperformance is concentrated in a narrow region of the tuning space, so the strategy should be interpreted as highly parameter-sensitive rather than uniformly dominant.

10.1 Learning-rate sensitivity

The one-dimensional learning-rate search shows that cumulative return is not a smooth or monotone function of the learning rate. For small learning rates, performance remains clustered near a common baseline. In the plot of cumulative return against learning rate, values at 10^{-3}, 3×10^{-3}, 10^{-2}, and 3×10^{-2} all lie close to approximately 6.4. This suggests that, at these settings, the optimizer is either moving very slowly through the feasible set or remaining near its initialization for much of the backtest. In other words, the optimization routine is not strongly altering the portfolio from one rebalance date to the next in a way that materially changes long-run realized performance.

This pattern changes sharply once the learning rate becomes large enough. At learning rate 0.1, cumulative return increases dramatically to approximately 28.16. However, when the learning rate is increased further to 1.0, cumulative return falls back to roughly 2.2. Thus, performance improves only after the step size becomes large enough to move the solution away from the near-static region, but collapses once the step size becomes so large that the optimization path appears unstable. This is important substantively. Since the portfolio is generated by a finite-step numerical routine rather than by an exact solver, the learning rate is not merely a computational detail. It directly affects the realized portfolio sequence and, therefore, the observed investment performance.

10.2 Joint search over learning rate and gradient steps

The two-dimensional grid over learning rate and number of gradient steps makes this dependence even clearer. Most combinations generate cumulative returns very close to the same baseline level, around 6.39 to 6.41. In particular, for learning rates 0.0003, 0.001, and 0.003, the cumulative return is essentially unchanged across all step counts considered. This suggests that, in these regions, increasing the number of gradient steps is not enough to produce a materially different portfolio policy because the step size itself is too small for the optimizer to leave the same broad neighborhood of the simplex.

At intermediate and higher learning rates, however, the number of optimization steps matters sharply. For learning rate 0.03, cumulative return is close to baseline for 100, 300, and 1000 steps, but then rises substantially to 20.40 at 3000 steps and further to 23.22 at 5000 steps. This suggests a delayed threshold effect: the optimizer requires enough iterations before it can move sufficiently far from its initial condition and produce a materially different set of portfolio weights.

For learning rate 0.1, the behavior is even more striking. Performance is close to baseline for 100 and 300 steps, rises sharply to 28.16 at 1000 steps, falls to 5.14 at 3000 steps, and then partially recovers to 9.71 at 5000 steps. This nonmonotone pattern indicates that the interaction between learning rate and iteration count is highly nonlinear. At some settings, the optimizer reaches a

region of the feasible set associated with strong realized performance. At nearby settings, it appears to overshoot, move into a poorer local configuration, or otherwise generate a materially different allocation path. This behavior underscores that the portfolio rule is highly sensitive not only to the economic objective, but also to how that objective is numerically traversed.

The best observed solver specification in the grid occurs at

$$\texttt{LR} = 0.1, \qquad \texttt{STEPS} = 1000,$$

which delivers total cumulative return 28.16. A second strong region occurs at

$$\texttt{LR} = 0.03, \qquad \texttt{STEPS} \in \{3000, 5000\},$$

which produces cumulative returns 20.40 and 23.22, respectively. By contrast, most other settings produce cumulative return around 6.4, implying that the difference between weak and strong performance is not gradual but rather concentrated in a relatively small part of the solver parameter space.

10.3 Interpreting the solver results

These solver results have a direct economic interpretation. When the economic penalties Γ and are very small, the optimizer is close to solving a constrained expected-return maximization problem. Over the long-only simplex, that kind of problem tends to favor concentrated allocations toward the assets with the highest estimated expected returns. If the learning rate is too small or the number of steps too low, the optimizer may fail to reach those concentrated regions. If the learning rate is too large, the update path may become unstable. The strongest performance therefore emerges only when the solver is aggressive enough to move the portfolio substantially, but not so aggressive that the optimization path becomes erratic.

In this sense, the numerical search is economically meaningful. It is not merely finding a faster way to solve the same portfolio problem. Instead, it is selecting among different approximate portfolio rules induced by different finite-step optimization dynamics.

10.4 Grid search over risk aversion and turnover penalty

The search over the economic parameters Γ and yields an equally strong and clearer economic pattern. The total cumulative return heatmap shows that the best outcomes occur when both penalties are small. The strongest specification is

$$\Gamma = 0.001, \qquad = 10^{-5},$$

which yields cumulative return 28.16. Very similar outcomes appear at

$$(\Gamma,) = (0.01, 10^{-5}) \quad \text{and} \quad (0.1, 10^{-5}),$$

with cumulative returns 28.13 and 27.80, respectively. This shows that performance remains very strong across a range of low-to-moderate variance penalties as long as the turnover penalty remains extremely small.

Once the turnover penalty increases, performance falls sharply. For example, when

$$= 0.001,$$

cumulative return declines into the range 23.81 to 24.20 for small to moderate Γ. When

$$= 0.01,$$

cumulative return falls further to around 7.30 to 7.45. Finally, when

$$= 0.1,$$

performance collapses to approximately 6.39 to 6.41 almost regardless of the value of Γ. This indicates that large turnover penalties effectively immobilize the strategy. Once deviations from the previous portfolio become expensive enough, the optimizer is no longer able to reallocate meaningfully toward the assets with the highest estimated means, and the resulting return path becomes close to the low-performance baseline.

The effect of risk aversion is somewhat more nuanced but still clear. For fixed small , moving Γ from 0.001 to 0.1 does not materially reduce cumulative return. However, once Γ becomes large, especially at

$$\Gamma = 1 \quad \text{or} \quad \Gamma = 10,$$

performance weakens substantially. For instance, at $= 10^{-5}$, cumulative return drops from above 27 for $\Gamma \in \{0.001, 0.01, 0.1\}$ to 18.13 at $\Gamma = 1$, and then to 4.23 at $\Gamma = 10$. Thus, strong variance penalization suppresses the aggressive concentration that appears to generate the strongest realized returns. This is consistent with the form of the objective: once the quadratic risk penalty becomes dominant, the optimizer is pushed away from concentrated, high-mean allocations and toward more conservative portfolios.

10.5 Economic interpretation of the penalty search

Taken together, the $(\Gamma,)$ search suggests that the strongest realized performance is obtained when the strategy is allowed to trade aggressively and is only weakly discouraged from holding concentrated positions. In practical terms, the best-performing portfolios arise in a region where the model behaves much more like a return-seeking allocator than like a classical mean-variance investor with substantial trading frictions.

This interpretation is important. The backtest does not suggest that stronger regularization improves the strategy. Rather, it suggests the opposite: the more the model is constrained by risk and turnover considerations, the more its realized performance reverts toward the low baseline region. The exceptional backtest outcomes therefore appear to come precisely from the aggressive edge of the admissible parameter space.